

Predicting the Targeted Industry of Cyber Threats

An End-to-End Machine Learning Project: From Raw Threat-Intelligence Data to a Deployed Web Application

Prepared by: Mohsin Khan

Domain: Cybersecurity / Threat Intelligence

Task Type: Multi-Class Classification

Live App: <https://spectacular-selkie-0cbeca.netlify.app>

Abstract / Executive Summary

This project builds and deploys a machine learning model that predicts the most likely targeted industry sector of a cyber threat-intelligence report. Using 2,365 real threat "pulses" from AlienVault OTX (April 2026, sourced via Kaggle), the raw multi-label, largely-unlabelled data was cleaned into 1,136 usable records spanning 11 industry classes. Report titles and tags were encoded with TF-IDF, country and malware-family fields were simplified and one-hot encoded, and two classifiers — Logistic Regression and Random Forest — were trained and compared under stratified 80/20 train-test splitting with class-weight balancing for the moderate class imbalance present. The final Random Forest model (300 trees, Tags + Title TF-IDF features) reached 43.9% accuracy and a weighted F1 of 0.412, roughly 4.8x better than the ~9% random baseline for an 11-class problem. The trained pipeline was serialised and deployed as a Flask REST API on Render, with a static HTML/JavaScript front end deployed on Netlify, giving a working public application where a user can enter report details and receive the top-3 most likely targeted industries with confidence scores. The full workflow — from dataset discovery through cleaning, encoding, modelling, evaluation, and deployment — was carried out with Claude (Anthropic) as a development assistant, documented in Section 11 and in the accompanying skills.md file.

1. Problem Definition

Cyber threat intelligence feeds (such as AlienVault OTX) publish thousands of unstructured threat reports every month. Analysts must manually read each report to work out which industry sector is being targeted, so they can route it to the right response team. This project builds a machine learning model that automates this triage step.

Real-world problem: Given the text and metadata of a newly published threat-intelligence report (title, tags, malware family, country of origin, description), automatically predict the most likely targeted industry sector, so security teams can prioritise and route reports faster.

Prediction target: `Industries_final` — the primary industry sector associated with a threat report (e.g. Government, Finance, Technology, Healthcare, Energy, or "Other" for long-tail sectors).

Input features: Report title, tags (keywords describing the attack), malware family involvement, country/origin of the report, and description length / tag count.

Task type: Supervised Machine Learning — Multi-Class Classification (11 target classes).

2. Dataset

The dataset used is the AlienVault OTX Threat Intelligence Pulses dataset, part of the "Cybersecurity Attacks & Defense Dataset 2026" collection published on Kaggle. It contains real threat-intelligence "pulses" (reports) contributed by AlienVault's global researcher community, including malware families, targeted countries, attack techniques (MITRE ATT&CK IDs), and industries.

Field	Detail
Publisher / Owner	Kaggle user "chuneeb", sourced from AlienVault OTX
Dataset name	Cybersecurity Attacks & Defense Dataset 2026 (OTX Threat Intelligence component)
URL	kaggle.com/datasets/chuneeb/ai-cybersecurity-threat-dataset-2026
Access date	August 9, 2026
License	As published on Kaggle (dataset available for research / educational use; see dataset page for full terms)
Rows (raw)	2,365 threat-intelligence pulses
Columns (raw)	14
Recency	Reports dated April 2026 — within the required 2025–2026 window

Key raw columns

- `Pulse_ID` — unique identifier for each threat report
- `Title, Description` — free-text summary of the threat/campaign
- `Tags` — comma-separated keywords describing the attack (e.g. "phishing, lumma stealer, powershell")
- `Malware_Families` — named malware/tooling involved, or "Unknown"
- `Attack_IDs` — MITRE ATT&CK technique IDs referenced in the report

- Industries — comma-separated list of industry sectors targeted (raw target source)
- Countries — comma-separated list of countries associated with the campaign
- TLP, Indicators_Count, Subscribers — metadata fields from the OTX platform

Note on dataset choice: cybersecurity threat-intelligence data is inherently "live" and constantly updated — there is no fixed, static canonical version the way there is for e.g. a census dataset. This snapshot (pulses through late April 2026) was the most recent credible extract available at the time of the project and satisfies the 2025–2026 recency requirement.

3. Data Exploration

Initial inspection used `df.shape`, `df.info()`, and `df.isnull().sum()` in Google Colab.

- Shape: 2,365 rows × 14 columns.
- No missing values in most columns except Description (1 null).
- Data types: mostly object (text/categorical); Indicators_Count and Subscribers were float64.

Key observations from EDA

A first attempt used a different, IP-address-based threat dataset ("Global Cyber Threat Intelligence Dataset 2026", 10,000 rows). EDA revealed that its label, risk_level, severity, abuse_confidence_score, time_zone_attack and reported_weekday columns were all constant (a single repeated value across all 10,000 rows), because the extract was a single-day snapshot. This meant no genuine classification target existed in that dataset, so the project switched to the OTX Pulses dataset described above — this pivot is itself documented here as part of the data-understanding process.

For the OTX dataset, `value_counts()` on candidate columns showed:

- TLP: 2,362 "white" vs 3 "green" — extremely imbalanced, not useful as a feature.
- Indicators_Count and Subscribers: constant at 0.0 for every row — dropped.
- Industries: 52% of rows were "Unknown"; the remaining rows were highly multi-label (469 unique combinations, e.g. "Finance, Technology").
- Countries and Malware_Families: also multi-label with a large "Unknown" share, requiring simplification.
- Description: text is truncated at ~300 characters in the raw extract, limiting its usefulness as a length-based feature.
- Duplicate check: 0 fully duplicate rows; 15 duplicate Pulse_IDs were found and handled during cleaning.

4. Data Cleaning

Issue found	Decision & rationale
52% of Industries values were "Unknown"	Rows with Industries = "Unknown" were removed, since there is no ground-truth label to train or evaluate against. Result: 1,136 usable rows.
Industries is multi-label (e.g. "Finance, Technology")	Simplified to a single primary label by taking the first listed industry (Industries_main), producing a clean single-class target per row — a documented simplification, not an attempt to solve multi-label classification.
469 unique industry labels after splitting, many with 1–2 rows	Grouped into the 10 most frequent categories plus an "Other" bucket (Industries_final), giving 11 well-populated classes suitable for classification and stratified splitting.

15 duplicate Pulse_IDs	De-duplicated on Title + Description (keep first occurrence) to avoid the same report being counted twice and leaking between train/test.
Constant / non-informative columns (TLP, Indicators_Count, Subscribers, Pulse_ID)	Dropped — a column with a single repeated value (or a pure identifier) carries no predictive signal and cannot be learned from.
Multi-label Countries field	Simplified to the first listed country (country_main), then grouped into the top 8 most frequent countries plus "Other" (country_final), mirroring the Industries treatment.
Multi-value Malware_Families with 288/1136 "Unknown" and hundreds of rare combinations	Converted to a binary flag has_known_malware (1 if any malware family was identified, 0 if "Unknown") rather than one-hot encoding hundreds of rare combinations.

5. Categorical Encoding

Different encoding strategies were used depending on the nature of each categorical feature:

Feature	Method	Why
country_final	One-Hot Encoding (pandas get_dummies)	Nominal categorical with no inherent order and a manageable number of groups (9) after top-N grouping.
has_known_malware	Binary flag (0/1)	Naturally binary after simplification — no further encoding needed.
Tags	TF-IDF vectorisation (top 30 terms, English stop-words removed)	Free-text, multi-keyword field; TF-IDF captures which keywords are most distinctive for a report without exploding into hundreds of rare one-hot columns.
Title	TF-IDF vectorisation (top 20 terms)	Same rationale as Tags; added in a second iteration to give the model more textual signal, which measurably improved performance (see Section 9).

The resulting feature matrix combines these encoded blocks with the two numeric features into a single matrix of 42 features (62 in the final version, after adding Title TF-IDF) across 1,136 rows.

6. Preprocessing

Scaling: The two numeric features (description_length, tags_count) were standardised with scikit-learn's StandardScaler (zero mean, unit variance), fitted on the training set only and applied to both train and test to avoid data leakage. The remaining features (one-hot flags and TF-IDF weights) are already on comparable [0,1]-ish scales and were left unscaled.

Class imbalance: The target is moderately imbalanced (Government: 371 rows vs Telecommunications: 24 rows). This was addressed by passing class_weight='balanced' to both models, which re-weights the loss function so minority classes are not ignored, rather than synthetically oversampling a text-derived feature set.

Reproducible pipeline: All preprocessing objects — the StandardScaler, both TfidfVectorizers, the list of top countries, and the final feature-column order — were fitted once on the training data and then serialised with joblib alongside the trained model, so the exact same transformations can be replayed at inference time in the deployed API (see Section 10).

7. Train / Test Split

The cleaned, encoded dataset (1,136 rows) was split 80/20 into training and test sets using scikit-learn's `train_test_split`, with `stratify=y` to preserve the class proportions of all 11 industry categories in both splits, and a fixed `random_state=42` for reproducibility.

Split	Rows	Proportion
Training set	908	80%
Test set	228	20%

Because the TF-IDF vectorisers and `StandardScaler` were fitted only on the training data (via `fit_transform` on train, `transform` on test), the split avoids data leakage from the test set into the preprocessing steps.

8. Model Development

Two model families were trained and compared, both with `class_weight='balanced'` to counter class imbalance:

Model 1 — Logistic Regression

```
LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
```

Model 2 — Random Forest (initial)

```
RandomForestClassifier(n_estimators=200, max_depth=10,  
                       class_weight='balanced', random_state=42)
```

Model 2b — Random Forest (final, with Title TF-IDF added)

```
RandomForestClassifier(n_estimators=300, max_depth=12,  
                       class_weight='balanced', random_state=42)
```

The Random Forest was chosen as the second model because the feature space mixes sparse TF-IDF text features with dense categorical/numeric features and non-linear class boundaries — an ensemble of decision trees typically handles this kind of heterogeneous, high-dimensional feature mix better than a linear model.

9. Evaluation

Accuracy and weighted F1-score were used as headline metrics, supplemented by a full per-class precision/recall/F1 report. Weighted F1 was prioritised over plain accuracy because the classes are imbalanced — a model that always predicted "Government" would still score ~33% accuracy without being useful.

Model	Accuracy	Weighted F1
Logistic Regression	25.4%	0.293
Random Forest (Tags only)	38.6%	0.382
Random Forest (Tags + Title) — FINAL	43.9%	0.412

Interpretation

- With 11 target classes, random guessing would score roughly 9% accuracy; the final model's 43.9% is roughly 4.8× better than chance, confirming it is learning a genuine signal rather than guessing.
- Random Forest clearly outperformed Logistic Regression, consistent with the mixed sparse/dense, non-linear feature space described in Section 8.
- Adding Title TF-IDF features (Model 2b) improved accuracy from 38.6% to 43.9%, showing the report title carries meaningful signal beyond the Tags field alone.
- Per-class performance is uneven: Government (F1 0.63) and Finance (F1 0.55) — the two best-represented classes with 296 and 186 training rows respectively — were predicted well. Energy and Telecommunications (F1 0.00) had only 24–30 total rows and were effectively unlearnable at this sample size.
- This pattern is a direct, expected consequence of class imbalance and overall dataset size (1,136 rows across 11 classes) rather than a modelling error — it is flagged here as an honest limitation rather than masked by a single aggregate accuracy figure.

10. Final Model & Deployment

Final model selected: Random Forest Classifier (n_estimators=300, max_depth=12, class_weight='balanced'), trained on Tags + Title TF-IDF features, one-hot country, and the has_known_malware / description_length / tags_count features — selected because it achieved the highest accuracy and weighted F1 of all models tested, and produces class-probability estimates (used to show a "top 3 likely industries" list in the app, which is more useful to an analyst than a single hard prediction).

Saved inference pipeline

To make the model reproducible for inference outside of the training notebook, all fitted preprocessing objects and the model itself were serialised with joblib:

- final_model.pkl — the trained Random Forest classifier
- scaler.pkl — fitted StandardScaler for the numeric features
- tfidf_tags.pkl, tfidf_title.pkl — fitted TF-IDF vectorisers
- top_categories.pkl, top_countries.pkl — the category groupings used during cleaning
- feature_columns.pkl — the exact final column order, so new inference rows can be reindexed to match training exactly

Deployment architecture

The trained pipeline is served through a small Flask REST API, and a static HTML/JavaScript front end calls that API to display predictions:

Component	Technology	Hosting
Model training	Python, scikit-learn, pandas (Google Colab)	—
Backend API (/predict endpoint)	Flask + Flask-CORS + gunicorn	Render (Web Service, free tier)
Frontend (form + results UI)	HTML / CSS / vanilla JavaScript (fetch API)	Netlify
Version control	Git / GitHub	github.com/Mohsinkhan3377/threat-industry-predictor

Live application: <https://spectacular-selkie-0cbeca.netlify.app>

Backend API: <https://threat-industry-predictor.onrender.com>

The front end collects a report title, tags, country, known-malware flag, and description, sends them as JSON to the /predict endpoint, and the Flask backend reconstructs the exact same feature vector used during training (scaling the two numeric features, one-hot encoding the country, and TF-IDF transforming the tags and title with the same fitted vectorisers) before calling `model.predict()` and `model.predict_proba()`. The API returns the top prediction plus the three most likely industries with their confidence scores, which the front end renders as a simple results panel.

Note: the backend is hosted on Render's free tier, which sleeps after a period of inactivity; the first request after idle time can take 30–60 seconds while the service wakes up. This is disclosed to the user in the app's loading state.

11. AI-Assisted Workflow

Claude (Anthropic) was used throughout this project as a pair-programming and workflow assistant, in the following ways:

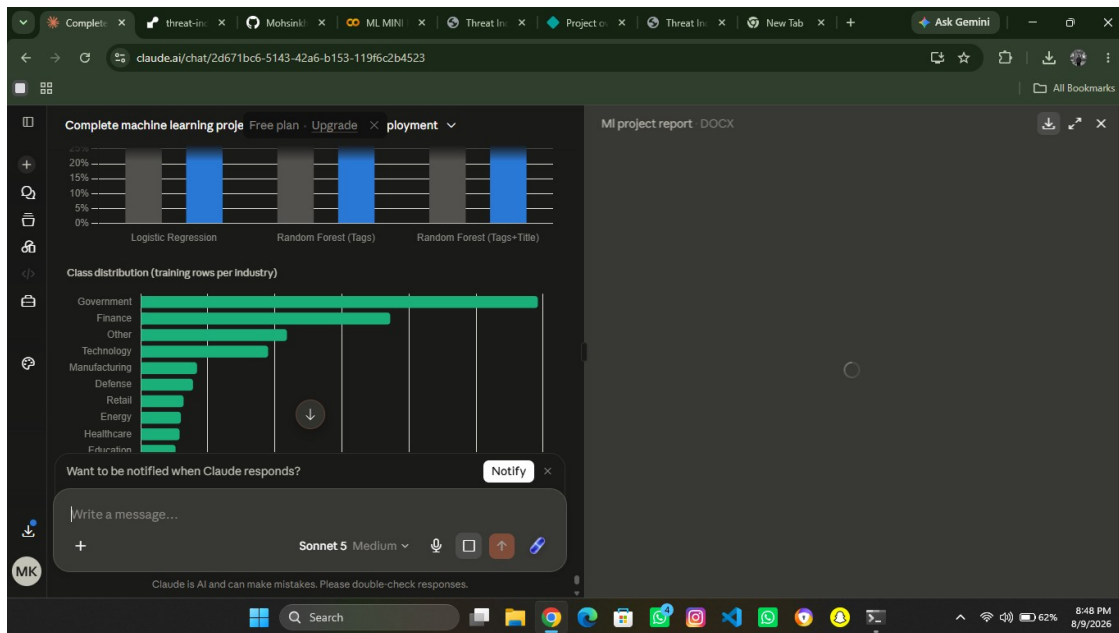
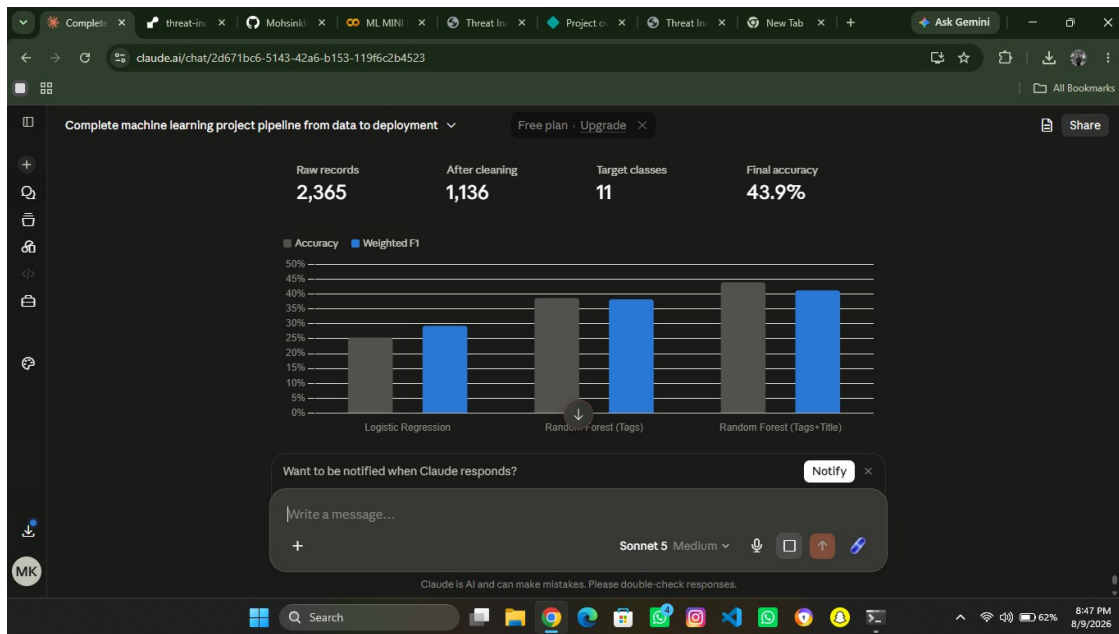
- Dataset discovery: web search was used to identify current (2025–2026) candidate cybersecurity datasets on Kaggle before one was selected.
- EDA guidance: Claude suggested and explained the pandas commands used to inspect shape, dtypes, missing values, and value distributions, and helped interpret the results (e.g. spotting that several columns in the first candidate dataset were constant).
- Cleaning & encoding decisions: Claude proposed and justified each cleaning step (Unknown removal, multi-label simplification, rare-category grouping, TF-IDF vs one-hot choice) rather than applying them silently.
- Model code: Claude wrote the scikit-learn training/evaluation code for both Logistic Regression and Random Forest, and helped interpret the classification reports.
- Backend/frontend development: Claude wrote the Flask API (`app.py`) and the HTML/CSS/JavaScript front end, including the logic to reconstruct the training-time feature vector at inference time.
- Deployment troubleshooting: Claude diagnosed and fixed several real issues encountered during deployment — a malformed multi-line terminal paste, Windows accidentally creating folders instead of files, a Git authentication flow, and a broken front-end file caused by a text-editor save issue — by asking for exact command output/error text and giving targeted fixes rather than restarting from scratch.

All modelling decisions, metric interpretations, and final architecture choices in this report reflect the actual outputs produced during this project's development, not hypothetical or idealised results.

12. Claude Artifact & Connector

Claude Artifact

An interactive dashboard was built with Claude as a companion visualisation for this report. It presents: (1) headline dataset statistics (raw record count, cleaned record count, number of target classes, final accuracy); (2) a grouped bar chart comparing accuracy and weighted F1 across the three models trained (Logistic Regression, Random Forest with Tags only, and the final Random Forest with Tags + Title); (3) a horizontal bar chart of training-set row counts per industry class, visually illustrating the class-imbalance issue discussed in Section 9; and (4) a direct link to the live deployed application. This gives a reviewer a single-glance summary of the project's data and results without needing to open the notebook or re-read the full metrics tables. Screenshots of this artifact are shown below.



Connector

This project's development workflow did not require a live data connector (e.g. Google Drive, Notion, or a CRM) because the dataset was obtained directly via the Kaggle API/CLI from within the Google Colab notebook, and the code/deployment pipeline was managed directly by the author through the GitHub CLI, Render, and Netlify dashboards. The closest analogue used in the Claude-assisted workflow was Claude's web search tool, used during Section 2 (dataset discovery) to identify current, credible 2025–2026 candidate datasets before one was selected — functioning as a live external information source feeding directly into a project decision, in the same spirit as a connector integrating an external service into the workflow.

13. Conclusion & Future Work

This project demonstrates a complete, reproducible machine learning pipeline: a real 2026 cybersecurity dataset was sourced, understood, and cleaned; text and categorical features were encoded appropriately; two models were trained, compared, and evaluated with metrics interpreted in context rather than reported blindly; and the final model was deployed as a working, publicly accessible web application with a properly separated frontend/backend architecture.

What was learned

- Real-world datasets frequently fail to match their apparent structure on first inspection — the first candidate dataset looked usable until `value_counts()` revealed several constant columns, reinforcing that EDA must precede any modelling decision, not follow it.
- Multi-label categorical fields (Industries, Countries, Malware_Families) require an explicit, documented simplification strategy; there is no single "correct" one-hot encoding for a field that doesn't fit the one-row-one-category assumption.
- Small, imbalanced multi-class datasets (1,136 rows across 11 classes here) impose a hard ceiling on achievable accuracy that no amount of model tuning alone can overcome — more data or fewer, better-populated classes would matter more than a fancier algorithm.
- Production deployment surfaces failure modes that never appear in a notebook — text-encoding mismatches, mismatched preprocessing artefacts between training and inference, and environment/runtime differences all had to be diagnosed from real server logs.

Limitations

- At 1,136 labelled rows across 11 classes, several minority classes (Energy, Telecommunications, Healthcare) have too few examples for the model to learn reliable patterns, visible in their near-zero F1 scores.
- The primary-label simplification of the Industries field discards genuine multi-label information — a report tagged "Finance, Technology" is only credited as "Finance", which likely both loses signal and mislabels some Technology-relevant reports.
- The Description field is truncated at ~300 characters in the source extract, limiting how much additional textual signal could be extracted from it.

Realistic future improvements

- Collecting or combining additional OTX pulses over a longer time window would directly address the small-sample-size limitation for minority classes.
- Reformulating the target as true multi-label classification (predicting all listed industries, not just the first) would better reflect the underlying data and could be evaluated with subset accuracy / label-based F1.
- Fetching the untruncated Description field (if available via the OTX API directly rather than the static Kaggle export) could add meaningfully more textual signal via TF-IDF or a lightweight transformer embedding.
- A gradient-boosted model (e.g. XGBoost/LightGBM) could be compared against the current Random Forest, particularly given the sparse TF-IDF + dense categorical feature mix.

14. References

- Dataset: AlienVault OTX Threat Intelligence Pulses, via "Cybersecurity Attacks & Defense Dataset 2026", Kaggle (published by user chuneeb). kaggle.com/datasets/chuneeb/ai-cybersecurity-threat-dataset-2026 — accessed August 9, 2026.
- scikit-learn — Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, 2011. scikit-learn.org

- pandas — McKinney, W., "Data Structures for Statistical Computing in Python", Proceedings of the 9th Python in Science Conference, 2010. pandas.pydata.org
- Flask — Pallets Projects. flask.palletsprojects.com
- Render (web service hosting). render.com
- Netlify (static site hosting). netlify.com
- Claude (Anthropic) — used throughout this project's development workflow as documented in [skills.md](#).